

Testing Standards — Interpop

Documento mestre de testes. Define os 10 tipos core praticados (§2.1-§2.10) + 13 tipos de extensão reconhecidos (§2.11) + protocolo formal para criar tipo novo (§2.12), stack, gates de cobertura, naming, templates de report e workflow. Referenciado pelo `AGENTS.md §6` como regra inegociável.

Versão 1.1 — 2026-05-21. Próxima revisão obrigatória: a cada novo ADR de teste em `Improvement-system.md §11.2` ou §12.1.

1. Princípio fundamental

Antes de implementar, modificar ou refatorar qualquer código que possa ser testado, deve existir teste. Sem exceção.

Justificativa de primeiro princípio:

- Código sem teste é hipótese, não asserção.
- Bug não testado volta. Sempre. O C1 (`rotate_refresh_token` quebrado) ficou anos despercebido porque nenhum teste exercitava o caminho — corrigido em `backend/apps/users/services.py:79` e protegido eternamente pelo regression test `backend/apps/users/tests/test_services.py`.
- Cobertura é o índice mais barato de saúde estrutural. Não é métrica vaidade.
- TDD não é cerimônia — é defesa contra a tentação de "depois eu testo".

2. Os 10 tipos de teste core praticados

2.1 Teste unitário

Definição. Exercita 1 função, método ou classe **isoladamente**. Sem DB, sem HTTP, sem filesystem, sem rede. Dependências externas são mocks ou stubs. Cada teste roda em milissegundos.

Quando usar.

- Lógica pura: validators, computed properties, helpers, utils.
- Cálculos de domínio: `engagement_rate`, `view_count` math, slug uniqueness.
- Transformações de dados: serializer field methods, normalizers.

Quando NÃO usar.

- Quando precisa validar fluxo HTTP completo — use **integração**.
- Quando precisa garantir que signal dispara — use **integração** com `@pytest.mark.django_db`.

Exemplo real no Interpop.

```
# backend/apps/users/tests/test_models.py
@pytest.mark.parametrize(
    'fixture_name,is_dev,is_admin,is_editor,can_publish,is_immune',
    [
        ('dev_user',    True,  True,  False, True,  True),
        ('admin_user',  False, True,  False, True,  True),
        ('editor_user', False, False, True,  True,  False),
        ('reader_user', False, False, False, False, False),
    ],
)
```

```
def test_role_properties_match_matrix(...):
    user = request.getfixturevalue(fixture_name)
    assert user.is_dev == is_dev
    assert user.is_admin == is_admin
    # ...
```

10 testes parametrizados cobrem a matriz inteira de role x property — sem tocar DB para validações puras.

2.2 Teste retroativo (backfill)

Definição. Teste escrito **depois** do código de produção, com objetivo de criar rede de proteção em área pré-existente sem cobertura. Não confundir com TDD (que vem antes do código). Não confundir com regressão (que reproduz bug específico).

Quando usar.

- Refactor de área crítica que nunca teve teste (ex.: substituir lógica de auth).
- Bug descoberto em código velho (escreve backfill que cobre o caminho antes do fix).
- Auditoria identifica gap de cobertura em módulo carregando lógica de domínio.

Anti-padrão. Escrever backfill por métrica de cobertura sem critério (ex.: copiar implementação no teste — circular). Backfill deve testar **comportamento**, não implementação.

Exemplo real no Interpop.

Quando setup pytest entrou (commit 992e0ca), todo o apps/users/services.py estava sem cobertura. O bloco JWT cookie flow (test_auth_flow.py) com 10 testes de integração HTTP via APIClient é backfill puro: cobre código que já existia (login, me, refresh, logout, register) sem mudar comportamento — só formalizando contrato.

2.3 Teste TDD (test-driven-development)

Definição. Ciclo **vermelho** → **verde** → **refactor**:

1. Escrever teste que falha (vermelho).
2. Escrever código mínimo que faz passar (verde).
3. Refatorar mantendo verde.

Teste **vem antes** do código de produção. Não é "escrever os dois juntos" — é vermelho explícito visto, depois código.

Quando obrigatório (não negociável):

Contexto	Por quê
Lógica de segurança (roles, permissions, auth)	C1 prova: bug crítico passou anos por falta de teste-first.
Cálculos de domínio (engagement_rate, view_count, audit metrics)	Math errado vira decisão editorial errada.
Migrations de dados (one-shot scripts)	Idempotência é difícil, regressão em prod é catastrófica.
Hotfix de produção	TDD reverso : reproduzir bug em teste ANTES do fix. Garante regressão zero.

Quando **NÃO** obrigatório:

- CRUD trivial (serializers básicos sem regra) — escrever teste **em paralelo** ou logo depois. TDD aqui vira ritual sem ROI.
- Componentes UI puramente visuais — usar **visual regression** (Chromatic/Percy) + vitest pra lógica.

Anti-padrão. "Escrever um teste rápido pra dizer que testou". Teste TDD deve falhar pelo motivo certo antes de passar. Se passa de primeira, ou (a) você testou a coisa errada, ou (b) o teste é tautológico.

2.4 Teste de integração

Definição. Múltiplos componentes interagem com **dependências reais** (DB, cache, fila — pode ser containers efêmeros do CI). Testa fronteiras: view → serializer → signal → service → ORM → DB.

Quando usar.

- API endpoints completos via DRF `APIClient`.
- Signals + service interagindo (ex.: `post_save` em Article dispara task de email).
- Migration scripts: aplicar em DB real anônimo e validar.
- Permissions matrix: bater no endpoint com cada role e validar HTTP status.

Estado no Interpop.  Ativo, parcial. `test_auth_flow.py` (10 testes) e `test_views.py` em articles (16 testes) já são integração — bate em URLs reais com `APIClient`, exercita middleware (axes, CSRF, auth), serializer, view, signal, DB.

Limitação atual. Sem Celery worker rodando em CI — tasks rodam em `ALWAYS_EAGER=True` (síncrono no request thread). Quando A20-A22 entrar em prod com Redis, adicionar integração assíncrona real (worker em container CI).

2.5 Teste E2E (end-to-end)

Definição. Fluxo completo: browser → frontend → API → DB → resposta visual. Simula usuário real. Roda em Chromium + Firefox + WebKit via **Playwright**.

Quando usar.

- Fluxos críticos: login, publicar artigo, comentar, banir.
- Validação de regressão UX/visual após refactor grande.
- Smoke test pós-deploy (canary).

Estado no Interpop.  **Implementação futura** — Sprint 3+. Stack planejada:

```
npm install -D @playwright/test
npx playwright install --with-deps chromium
```

Estrutura prevista:

```
e2e/
├─ playwright.config.ts
├─ fixtures/
│   └─ users.ts           (logged-in fixtures)
├─ flows/
│   ├── auth.spec.ts      (login, register, password reset)
│   ├── editorial.spec.ts (publicar, editar, deletar artigo)
│   ├── reader.spec.ts    (ler, comentar, curtir)
│   └─ admin.spec.ts      (banir, ver métricas)
└─ visual/
    └─ snapshots/         (screenshots de regressão visual)
```

Cronograma: Sprint 3 — `e2e/auth.spec.ts` + `e2e/editorial.spec.ts` (4-5 testes cada cobrindo golden path). Sprint 4 — expandir + visual regression.

Custo previsto. Playwright pesa ~120 MB (binário Chromium). Roda só em PR pra `main` (não em todo push) — caro pra rodar em todo commit.

2.6 Teste de regressão

Definição. Teste escrito **após um bug**, reproduzindo o cenário exato que falhou em produção. Único propósito: garantir que o mesmo bug **nunca volta**.

Workflow obrigatório:

1. Bug é reportado / descoberto.
2. Antes de escrever fix, escreve teste que **reproduz o bug e falha**.
3. Aplica fix.
4. Teste agora passa.
5. Commit do fix junto com o teste (atomic).
6. Mensagem do commit referencia o ID do bug + caminho do teste.

Estado no Interpop.  Ativo. Exemplos reais:

- **C1** (`rotate_refresh_token` quebrado): regression em `test_services.py::test_rotate_refresh_token_returns_true_for_valid_cookie`. Mensagem de assertion diagnóstica aponta pra causa raiz.
- **C2** (signal duplicado de email): regression em `test_views.py::test_article_publish_triggers_send_article_notification_once`. Mock confirma `call_count == 1` (era 2).
- **C3** (transação atomic): regression em `test_services.py::test_ban_user_rollback_on_failure`. Mock força falha no segundo write e valida rollback.
- **C4** (anti-abuse view_count): regression em `test_views.py::test_view_count_incremented_once_per_5min_window`. 3 POSTs do mesmo IP em sequência → counter = 1.

2.7 Teste de fumaça (smoke test)

Definição. Teste **mínimo e rápido** que valida apenas que o sistema **subiu** e responde no básico. Não testa lógica de domínio. Roda em segundos.

Quando usar.

- Pós-deploy automatizado (rollback se falha) — já implementado em `scripts/deploy.sh` (planejado §A.2 do `HOSTING-DEPLOY-PLAN.md`).
- Pós-restart de unicorn/celery (validação que nada quebrou na config).
- Health check externo (UptimeRobot bate `/healthz/` a cada minuto).

Estado no Interpop.  Ativo. Implementação:

- Endpoint `GET /healthz/` responde `{"status":"ok","db":"ok","cache":"ok"}` em <50ms.
- 4 testes formais em `apps/audit/tests/test_health.py` validam 200 quando ok, sem auth, GIT_SHA truncado.
- `deploy.sh` (planejado) curl no `/healthz/` pós-restart com rollback automático se falhar.

Diferença vs unit/integration: smoke tem propósito **operacional** (sistema vivo?), não **funcional** (lógica correta?). Pode coexistir com os outros.

2.8 Teste de acessibilidade (a11y)

Definição. Valida conformidade com **WCAG 2.2 AA** (regra dura do `AGENTS.md §4`): contraste, navegação por teclado, semantic HTML, ARIA, focus order, screen reader compatibility.

Quando usar.

- Antes de submeter qualquer mudança visual de frontend (`AGENTS.md §4`).
- Após adicionar componente novo de UI ou alterar layout.
- Auditoria periódica (mensal) de páginas críticas.

Estado no Interpop.  Ativo (manual) — automação no roadmap.

- **WAVE** (extensão Firefox/Chrome) — relatório AIM Score, último: **10/10** (commit `6bde9ed` , 2026-05-21).
- **axe DevTools** — extensão browser, mais profundo que WAVE.
- **Firefox built-in Accessibility audit** — auditor nativo.

Roadmap automação (Sprint 3+):

- `@axe-core/playwright` em E2E tests — assertions automatizadas em cada fluxo.
- Lighthouse CI com gate `accessibility >= 95` .
- Auditoria trimestral com NVDA + Orca (manual, 5 fluxos críticos).

Política de violação. Falha automática em CI quando entrar gate axe-core. Hoje (manual): violação WCAG AA = blocker de PR.

2.9 Teste de performance

Definição. Mede **velocidade e eficiência** sob carga. 3 categorias:

- **Frontend** — Core Web Vitals (LCP, INP, CLS) via Lighthouse.
- **Backend** — latência por endpoint (p50/p95/p99) + número de queries.
- **DB** — query plan analysis (EXPLAIN), índices, N+1 detection.

Quando usar.

- Antes de qualquer mudança que afete request hot path (view, serializer, signal).
- Antes de adicionar nova lib pesada ao bundle frontend.
- Quando perfil de uso muda (ex.: artigo viralizando).

Estado no Interpop.  Parcial — base implementada, gates automatizados em roadmap.

- **Performance budgets** documentados em `Improvement-system.md §12.3.1-3` e `HOSTING-DEPLOY-PLAN.md §A.4` : LCP <1.8s home, p95 backend <300ms, queries ≤5 por endpoint público.
- **Lighthouse** rodando manual no Chrome DevTools.
- **Sentry Performance Monitoring** (A28) ativo — captura `traces_sample_rate=0.1` em produção.

Roadmap automação (Sprint 3+):

- Lighthouse CI com gates (`performance ≥ 85` , `LCP ≤ 1.8s` , `CLS ≤ 0.05`).
- `pytest-benchmark` para asserts de latência (`AdminMetricsView < 200ms`).
- `django-silk` em staging para detectar N+1 antes de PR.
- `assertNumQueries(N)` em testes de view.

2.10 Teste de segurança

Definição. Procura **vulnerabilidades** ativamente: SQL injection, XSS, auth bypass, secrets vazados, dependências vulneráveis, configuração insegura.

Quando usar.

- Em **todo PR** (SAST automatizado).
- Antes de release (pentest manual quando virar dor).
- Após CVE divulgada em dependência usada.

Estado no Interpop. 🟡 Parcial — fundação pronta, automação em CI no roadmap S15-S17 (`Improvement-system.md §11.6`).

- **Tests existentes** em `apps/moderation/tests/test_serializers.py` cobrem defesa em profundidade (BanSerializer rejeita dev/admin).
- **JWT cookie flow** testado E2E em `apps/users/tests/test_auth_flow.py`.
- **CSP, HSTS, security headers** documentados em `production.py`.

Roadmap automação (Sprint 1-2):

- **SAST:** `bandit -ll` + `semgrep --config p/django` + `semgrep --config p/javascript` em CI.
- **Secret scanning:** `gitleaks` em pre-commit + CI.
- **Dependency audit:** `pip-audit` + `npm audit` em CI semanal + Dependabot.
- **DAST** (futuro): OWASP ZAP contra staging.
- **Pentest manual** (futuro): B4 do backlog — contratado após 5k MAU.

Política dura. PR não merge se SAST acha severity ≥ MEDIUM. Secret detectado → block + rotação obrigatória.

2.11 Outros tipos de teste reconhecidos (catálogo de extensão)

Tipos consagrados na indústria que **ainda não são praticados** no Interpop mas podem entrar conforme dor surgir. Listados aqui para que decisões futuras de "criar tipo novo" não inventem nome quando já existe convenção.

Tipo	Definição curta	Quando virar dor no Interpop
Property-based testing (PBT)	Gera centenas de inputs aleatórios validando invariantes (Hypothesis em Python, fast-check em JS)	<code>_unique_slug</code> collision — tests #18-19 da lista §12.1.8
Snapshot test	Captura output e compara com snapshot fixo (revisa diff manualmente)	<code>renderArticleBody</code> com input fixo — paridade preview/leitura
Mutation testing	Mede QUALIDADE dos testes mutando o código de produção (Stryker, mutmut)	Sprint 3+ pra validar que testes não são tautológicos
Migration test	Aplica migrations em DB anônimo derivado de prod, valida reversibilidade	Antes de cada migration crítica que toca produção
Contract test	Valida que frontend ↔ backend mantêm shape acordado (OpenAPI schema diff, Pact)	A7 drf-spectacular + F9 openapi-typescript do Sprint 4
Visual regression	Captura screenshots e diff visual (Chromatic, Percy, BackstopJS)	Sprint 4 quando design system maduro
Load/stress test	Simula carga concorrente alta (k6, Locust, JMeter)	Antes de campanha de growth ou primeiro viral previsto
Fuzz testing	Bombardeia inputs malformados pra achar crashes (radamsa, AFL, atheris)	Validação real de upload (S5) — campos que aceitam binário
Concurrency / race condition	Testa comportamento sob threads/processos concorrentes (<code>pytest-asyncio</code> , <code>threading</code>)	Quando Celery worker real + multiprocesso entrar em prod

Tipo	Definição curta	Quando virar dor no Interpop
Doctest	Valida exemplos em docstrings (<code>pytest --doctest-modules</code>)	Documentação de API pública / lib publicada
Compatibility test	Cross-browser, multi-version (Python/Node matrix)	Antes de drop de suporte a versão
Localization test (l10n)	Valida traduções, formato de datas/números	Quando B15 i18n backend entrar
Acceptance test / BDD	Cenários em Gherkin (pytest-bdd, Behave)	Quando stakeholder não-técnico escrever requisitos

2.12 Quando criar tipo novo (regra de extensão)

Princípio. Se nenhuma das categorias 2.1-2.11 cobre o caso, **crie tipo novo** seguindo este protocolo — não use ad-hoc.

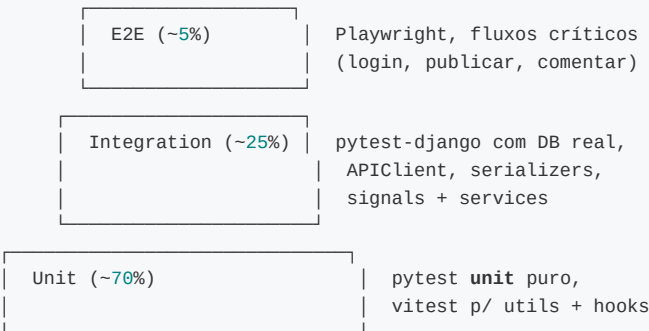
Protocolo:

1. **Justifique a necessidade** em 1 parágrafo: por que os tipos existentes não cobrem?
2. **Pesquise nome consagrado** na literatura (Test Pyramid de Cohn, ISTQB glossary, Google Testing Blog) antes de inventar.
3. **Crie ADR novo** em `Improvement-system.md §11.0` documentando o tipo (definição, quando usar, quando não, exemplo, estado, anti-padrão).
4. **Atualize este documento** adicionando seção §2.N+1 (estilo §2.1-2.10).
5. **Atualize** `AGENTS.md §6.1` adicionando linha na tabela de tipos core.
6. **Implemente pelo menos 1 exemplo** no projeto antes de fechar o PR — sem exemplo real, vira documentação morta.

Anti-padrão: criar "Smoke + integration" como tipo híbrido novo. Resposta: ou é smoke (5 linhas, /healthz) ou integration (DB real, fluxo). Pode coexistir, não fundir.

3. Pirâmide de testes

Distribuição alvo do número de testes (não de cobertura — distribuição):



Anti-padrão a evitar: pirâmide invertida (90% E2E, 10% unit) — lenta, instável, mascara bugs.

Estado atual (2026-05-21): 88 testes backend, ~70% unitários (test_models.py + test_services.py), ~30% integração (test_auth_flow.py + test_views.py). Zero E2E (planejado Sprint 3+). **Distribuição saudável** dentro da pirâmide.

4. Gates de cobertura evolutivos

Sprint	Gate backend	Gate frontend	Mutation score (PIT/Stryker — opcional)
1	≥40%	— (setup)	—
2	≥55%	≥30%	—
3	≥70%	≥50%	≥40% em módulos críticos
4	≥75%	≥60%	≥50%
pós-Sprint 4	≥80% (long-tail)	≥65%	≥60%

Estado atual: 82% backend (gate Sprint 4 já atingido), 0% frontend (vitest não setup ainda — entra Sprint 2 via F4).

Política dura: PR não merge se cobertura DESCE. Aceitável: cobertura sobe ou estável (mudanças não-funcionais como rename, doc).

Configurado em `.github/workflows/ci.yml`:

```
- run: uv run pytest --cov=apps --cov-fail-under=40
```

Sobe o número conforme sprint avança.

5. Stack ativa

5.1 Backend (Python + Django + DRF)

Lib	Versão	Função
pytest	9.x	Runner
pytest-django	4.12+	Fixtures DB, <code>client</code> autenticado, <code>--reuse-db</code>
pytest-cov	7.x	Cobertura via <code>coverage.py</code>
factory-boy	3.x	Fábricas substituindo <code>seed_users.py</code>
freezegun	1.5+	Congelar <code>timezone.now()</code> em testes de TTL
pytest-mock	3.x	<code>mock.patch</code> cleaner que <code>unittest.mock</code>
hypothesis	(futuro)	Property-based testing (<code>_unique_slug</code> collision)
pytest-benchmark	(futuro)	Não-funcional: AdminMetricsView <200ms
pytest-xdist	(futuro)	Paralelismo (<code>-n auto</code>)

Comandos:

```
cd backend
uv run pytest           # suite completa
uv run pytest -v        # verboso
```



```
uv run pytest apps/users/tests/ -k role # filtro
uv run pytest --cov=apps --cov-report=html # cobertura visual em htmlcov/
uv run pytest --reuse-db # reusa DB entre runs (5x mais rápido)
uv run pytest --create-db # força recriar (após migration)
```

5.2 Frontend (TypeScript + React) — futuro Sprint 2

Lib	Versão alvo	Função
<code>vitest</code>	1.x	Runner Vite-native (5× mais rápido que Jest)
<code>@vitest/coverage-v8</code>	1.x	Cobertura via V8
<code>@testing-library/react</code>	14+	Query-by-role, evita testes acoplados a implementação
<code>@testing-library/user-event</code>	14+	Simulação de eventos do usuário
<code>@testing-library/jest-dom</code>	6+	Custom matchers
<code>msw</code>	2.x	Mock Service Worker (intercepta axios em tests + dev)

5.3 E2E — futuro Sprint 3

- `@playwright/test` — Chromium + Firefox + WebKit.
- `@axe-core/playwright` — a11y assertions em E2E.

6. Convenção de naming e locality

6.1 Locality (onde colocar)

- Testes vivem **dentro do app** (Django convention): `backend/apps/<app>/tests/test_*.py`.
- 1 arquivo de teste = 1 unidade de produção: `models.py` → `test_models.py`, `services.py` → `test_services.py`, `views.py` → `test_views.py`, `serializers.py` → `test_serializers.py`.
- Factories ficam em `backend/apps/<app>/tests/factories.py` (quando precisar de factory complexa além do `conftest.py` global).
- Fixtures compartilhadas vão em `backend/conftest.py` (global) ou `backend/apps/<app>/tests/conftest.py` (escopo de app).
- Frontend (futuro): `src/**/__tests__/*.test.tsx` ao lado do componente (Vitest convention).

6.2 Naming (como nomear)

Padrão `test_<unidade>_<comportamento>_<contexto>`:

✓ Bom:

- `test_ban_user_idempotent_reactivates_existing`
- `test_rotate_refresh_token_returns_false_when_user_deleted`
- `test_view_count_incremented_once_per_5min_window`
- `test_role_properties_match_matrix[dev-user-True-True-False-True-True]` (parametrize gera sufixos automáticos)

✗ Ruim:

- `test_ban` (qual aspecto?)
- `test_works_correctly` (sempre "works"; o que muda?)
- `test_1`, `test_user_stuff` (cargo cult)

Regra geral: **se o nome não diz O QUE quebra quando falha, refaz o nome.**

7. Workflow padrão por tipo de teste

7.1 TDD (vermelho → verde → refactor)

```
# 1. Escrever teste primeiro
cd backend
$EDITOR apps/users/tests/test_services.py
# adicionar test_new_feature_does_x

# 2. Rodar — confirmar vermelho com motivo CERTO
uv run pytest apps/users/tests/test_services.py::test_new_feature_does_x -v
# FAIL: AssertionError ou ImportError (esperado)

# 3. Escrever código mínimo
$EDITOR apps/users/services.py
# implementar a função

# 4. Rodar — confirmar verde
uv run pytest apps/users/tests/test_services.py::test_new_feature_does_x -v
# PASSED

# 5. Refactor com teste verde como rede
# editar implementação visando clareza; teste continua verde

# 6. Commit atomic
git add apps/users/services.py apps/users/tests/test_services.py
git commit -m "Feat: nova feature X (TDD)"
```

7.2 Regressão (após bug)

```
# 1. Bug reportado: rotate_refresh_token retorna False sempre
# Diagnóstico já feito — sabe o motivo (AttributeError silencioso)

# 2. Escrever teste que REPRODUZ o bug
$EDITOR apps/users/tests/test_services.py
# adicionar test_rotate_refresh_token_returns_true_for_valid_cookie
# Roda → FAIL (confirmando bug)

# 3. Aplicar fix
$EDITOR apps/users/services.py
# substituir refresh.access_token.user por refresh['user_id']

# 4. Rodar — confirmar verde
uv run pytest apps/users/tests/test_services.py::test_rotate_refresh_token_returns_true_for_valid_cookie -v
# PASSED

# 5. Commit atomic — fix + regression test juntos
git commit -m "Fix: rotate_refresh_token quebrado (C1) + regression test"
```

7.3 Backfill (cobertura retroativa)

```
# 1. Escolher módulo crítico sem cobertura
uv run pytest --cov=apps/users --cov-report=term | grep services
# apps/users/services.py: 0%

# 2. Mapear comportamento via leitura do código (não cole código no teste)
# Identificar: input → side effect / output / state change

# 3. Escrever testes que validam comportamento OBSERVÁVEL
# (não implementação – backfill não deve quebrar em refactor)

# 4. Rodar até cobertura >= meta
uv run pytest --cov=apps/users --cov-report=term
```

8. Lista de testes prioritários (Sprint 1+)

Vivem em `Improvement-system.md §12.1.8` — lista canônica. Cópia condensada por referência:

Bloco	Total	Status atual
auth/role (is_admin, is_dev, can_publish, etc.)	1-5	✓ 10 testes
ban_user idempotência + workflow BanRequest	6-13	✓ 8 testes
JWT cookie flow (login → me → refresh → logout)	14-17	✓ 10 testes
<code>_unique_slug</code> collision + property-based via Hypothesis	18-19	🕒 pendente
Article publish dispara EXATAMENTE 1 email task	20-22	✓ 2 testes (test_views.py)
<code>Comment.soft_delete</code>	23-25	🕒 pendente
<code>PasswordResetToken</code> expira em 1h + single-use	26-27	🕒 pendente
axes bloqueia após 5 falhas + throttle 429	28-29	🕒 pendente
AuditLog criado em ações sensíveis	30	🕒 pendente

Atualizar este documento e §12.1.8 do `Improvement-system.md` quando completar cada bloco.

9. Convenção de reports de execução

9.1 Quando gerar report

- **Sempre** após uma execução de teste relevante: fim de sprint, validação pós-deploy, retest após refactor grande, auditoria solicitada.
- **Não** gerar pra cada `pytest` local de desenvolvimento — só quando o resultado é evidência arquivada.

9.2 Onde salvar

```
docs/tests/
├─ reports/
```

← markdown editável

```
| └─ 2026-05-21_16-38-42.md
└─ reports-pdf/
   └─ 2026-05-21_16-38-42.pdf
```

← cópia PDF (gerada via script)

9.3 Formato de timestamp

AAAA-MM-DD_HH-MM-SS — ISO 8601 com underscore separando data de hora, hífen dentro (não : porque Windows reclama). Fuso UTC-3 (Brasília).

```
# Comando para gerar timestamp correto
date + '%Y-%m-%d_%H-%M-%S'
# 2026-05-21_16-38-42
```

Ordenação alfabética = ordenação cronológica (propriedade-chave do ISO 8601).

9.4 Template do report (.md)

```
# Test Report — Interpop

**Data:** 2026-05-21 16:38:42 BRT
**Sprint:** 1
**Stack:** pytest 9.0 + pytest-django 4.12 + pytest-cov 7
**Branch:** develop @ commit `6bde9ed`
**Executor:** [nome ou CI run ID]

---

## Resumo simples + direto

[1-2 parágrafos curtos, qualquer leitor entende:]

- Quantos testes rodaram, quantos passaram, cobertura geral.
- Houve regressão? Sim/Não.
- Próxima ação humana (se houver).

---

## Análise técnica + detalhada

### Execução

```bash
cd backend
uv run pytest --cov=apps --cov-report=term
```

### Resultado bruto

```
===== 88 passed in 9.51s =====
TOTAL 2068 390 81%
```

### Cobertura por módulo



Módulo	Cobertura	Variação
apps/users/services.py	100%	=
apps/moderation/services.py	100%	=
...	...	...



### Testes adicionados nesta execução
```

```
- `apps/users/tests/test_models.py::test_role_properties_match_matrix` (parametrize × 4)
- ...

### Testes removidos

- `apps/users/tests/test_smoke.py` (substituído por testes substantivos)

### Falhas / warnings notáveis

- 28 warnings (não-bloqueadores): `vlibras-plugin.js` legacy CSS, paginação não-ordenada (a fixar).

### Comandos de reprodução

```bash
git checkout 6bde9ed
cd backend && uv sync --frozen
uv run pytest --cov=apps --cov-report=html
firefox htmlcov/index.html
```

### Próximas ações sugeridas

1. ...
2. ...
```

9.5 Geração do PDF

Script `scripts/md-to-pdf.sh` (a criar):

```
./scripts/md-to-pdf.sh docs/tests/reports/2026-05-21_16-38-42.md
# Gera docs/tests/reports-pdf/2026-05-21_16-38-42.pdf
```

Implementação via `pandoc` (instalar com `sudo apt install pandoc texlive-xetex texlive-fonts-recommended`).

Alternativa leve sem TeX: `npx mdpdf input.md output.pdf` (qualidade visual menor mas zero setup).

10. Anti-padrões a evitar

| Anti-padrão | Por que é ruim | Como evitar |
|---|---|--|
| Teste que testa o framework
(<code>assert User.objects.all()</code>) | Não pega bug do seu código | Testar regras de NEGÓCIO, não comportamento conhecido do Django |
| Mock excessivo (mockar tudo, inclusive ORM) | Testa o mock, não o código | Usar fixtures de DB reais via <code>@pytest.mark.django_db</code> |
| Snapshot de página inteira | Frágil, vira poluição visual no PR | Snapshot só de renderers determinísticos
(<code>renderArticleBody</code>) |
| Teste que só passa em ordem específica | Acoplamento via DB state ou import side-effect | Cada teste cria seu próprio estado; <code>--randomly-seed</code> no CI |
| Cobertura como métrica vaidade | 100% cobertura com testes triviais não pega bug | Mutation testing (Stryker) revela testes vazios |

| Anti-padrão | Por que é ruim | Como evitar |
|---|--------------------------------------|--|
| Compartilhar dados entre testes (variável global) | Testes ficam dependentes | Fixtures de pytest com escopo <code>function</code> |
| Comentar teste que falha pra "destravar PR" | Acumula dívida que ninguém vai pagar | <code>pytest.mark.xfail(reason=...)</code> com prazo de validade |

11. Roadmap de evolução

| Quando | O que entra | Tipo | Comentário |
|--------------|---|---------------------------|---|
| Sprint 2 | vitest + Testing Library + msw | Unit/Integration frontend | F4 do Improvement-system §11.3 |
| Sprint 2 | Hypothesis para <code>_unique_slug</code> collision | Unit property-based | Test #18-19 da lista §12.1.8 |
| Sprint 3 | <code>@playwright/test</code> setup + 4 fluxos críticos | E2E | §12.1.4 |
| Sprint 3 | Mutation testing piloto (Stryker em 1 módulo) | Meta-test | Validar qualidade dos testes existentes |
| Sprint 4 | Contract testing via OpenAPI schema | Integration cross-stack | A7 + F9 do §11.2/11.3 |
| Sprint 4 | Visual regression (Chromatic/Percy) | E2E visual | Componentes do design system |
| Pós-Sprint 4 | Load testing (k6 ou Locust) | Performance | Capacity validation pré-viral |
| Pós-Sprint 4 | Chaos engineering trimestral | Resiliência | §12.2.4 — postmortem template |

12. Referências externas canônicas

- [Test Driven Development by Example — Kent Beck](#) — origem do TDD.
- [pytest documentation](#) — runner oficial.
- [pytest-django docs](#) — fixtures e marks.
- [factory_boy docs](#) — fábricas.
- [Hypothesis docs](#) — property-based testing.
- [Testing Library principles](#) — filosofia query-by-role.
- [Playwright docs](#) — E2E.
- [roadmap.sh — testing](#) — alinhamento mainstream (referenciado em `AGENTS.md §5`).

13. Cross-refs canônicos do projeto

- `AGENTS.md §6` — política inegociável (resumo).

- `Improvement-system.md §12.1` — cultura de testes (rationale completo, pirâmide, gates).
 - `Improvement-system.md §11.2 A24-A26` — backlog setup pytest (concluído).
 - `HOSTING-DEPLOY-PLAN.md §A.2` — TDD na pipeline (status checks de merge gate).
 - `docs/tests/reports/*.md` — histórico de execuções.
-

v1.0 — 2026-05-21. Documento criado a partir do agregado de §12.1 do `Improvement-system.md` + práticas observadas nas 88 testes implementados nos últimos commits + os 6 tipos de teste explicitados em `AGENTS.md §6`. Próxima revisão: junto com cada novo ADR de teste.